

Campus Trading Application: Sharding Implementation Report

CS 432: Databases — IIT Gandhinagar Group 8 – Optimiser

Bhavik Patel
22110047
IIT Gandhinagar

Hitesh Kumar
22110098
IIT Gandhinagar

Jinil Patel
22110184
IIT Gandhinagar

Pranav Patil
22110199
IIT Gandhinagar

Sibtain Raza
22110148
IIT Gandhinagar

Abstract

This report details the implementation of Assignment 4 for the *Campus Trading Application* (CampusTradingB), extending the existing MySQL 8.0 / Go / React stack with horizontal scaling through database sharding. We describe four sub-tasks: (1) shard key selection using primary-key modulo routing across three shards; (2) data partitioning via hash-style modulo placement, with reference tables replicated and transactional tables partitioned; (3) query routing implemented in Go with direct routing for single-record lookups and fan-out merging for browse-style queries; and (4) a scalability and trade-offs analysis covering consistency, availability, and partition tolerance. The verified shard distribution is balanced, with listing rows distributed 41/42/42 and transaction rows 40/41/40 across the three shards, confirming that no data was lost or duplicated during migration.

GitHub Repository

Video Demonstration Link

Keywords

Database Sharding, Horizontal Scaling, Query Routing, MySQL, Go, Modulo Partitioning, Distributed Databases

1 SubTask 1 — Shard Key Selection & Justification

1.1 Chosen Shard Key

The project uses a **primary-key modulo** shard key with the routing rule:

$$\text{shard_id} = \text{record_id} \bmod 3$$

This applies to every partitioned table: the primary key of each row determines which of the three shard databases stores that row.

1.2 Justification Against the Three Criteria

Criterion	How the chosen key satisfies it
High Cardinality	Primary keys are unique auto-increment integers. Every table has a distinct value per row, giving the maximum possible spread across the value space.
Query-Aligned	The most common API operations — fetching a listing, reading an offer, checking a notification — all filter by a single primary-key ID. The shard key matches the dominant query pattern exactly.
Stable	Primary keys are assigned on insertion and never change. Routing therefore stays consistent for the lifetime of each record.

Modulo routing also gives a deterministic, $O(1)$ shard lookup with no external directory or lookup table required.

1.3 Expected Data Distribution and Skew Analysis

Consecutive auto-increment IDs distribute evenly under modulo-3 partitioning: $\{1, 4, 7, \dots\} \rightarrow \text{shard } 0$, $\{2, 5, 8, \dots\} \rightarrow \text{shard } 1$, $\{3, 6, 9, \dots\} \rightarrow \text{shard } 2$. For a table with N rows, each shard holds approximately $\lfloor N/3 \rfloor$ or $\lceil N/3 \rceil$ rows, giving at most one row of imbalance across any pair of shards.

The verified post-migration counts confirm near-perfect balance:

Table 1: Verified post-migration row distribution (output of `verify_shards.py`)

Table	Source	S0	S1	S2	Total	Duplicates
<i>Partitioned tables</i>						
Listing	125	41	42	42	125	0
Offer	124	41	42	41	124	0
Transaction	121	40	41	40	121	0
Message	44	14	15	15	44	0
MessageThread	22	7	8	7	22	0
Rating	22	7	8	7	22	0
ListingImage	25	8	9	8	25	0
Notification	25	8	9	8	25	0
Watchlist	24	8	8	8	24	0
WishRequest	14	4	5	5	14	0
Report	12	4	4	4	12	0
Member	5	1	2	2	5	0
<i>Replicated tables (identical copy on every shard)</i>						
Category	15	15	15	15	—	0
Administrator	1	1	1	1	—	0
all_passed: true no data loss zero duplicates						

⁰GitHub: https://github.com/Zeeun03/Campus_Trading_App

Skew Risk

The modulo key performs well for sequential IDs. Two scenarios where skew can occur are: (1) **Bulk deletions** — removing records from one shard's ID range can shift the balance over time, though this risk is low because campus marketplace data is rarely bulk-deleted. (2) **Non-sequential seeding** — if IDs are seeded with large gaps (e.g., only even IDs exist), one shard could receive no rows. The current seeder produces gapless consecutive IDs, so this risk is not present in the current data set.

2 SubTask 2 — Data Partitioning

2.1 Partitioning Strategy

The project uses **hash-style modulo partitioning** rather than range-based or directory-based partitioning.

This was the best fit for the assignment because:

- It is deterministic and O(1) for any known ID.
- It avoids the hot-spot skew that range partitioning produces when high-traffic records cluster at one end of the ID space.
- It matches the course requirement to simulate sharding using multiple databases on the same server without a separate lookup directory.

2.2 Shard Databases

Three shard databases are created on the same MySQL server:

Listing 1: Shard database creation

```
1 CREATE DATABASE IF NOT EXISTS CampusTradingB_shard_0;
2 CREATE DATABASE IF NOT EXISTS CampusTradingB_shard_1;
3 CREATE DATABASE IF NOT EXISTS CampusTradingB_shard_2;
```

2.3 Table Placement

Each table is classified as either *partitioned* (rows split across shards by modulo key) or *replicated* (all rows copied to every shard). The placement is declared in the Go routing map:

Listing 2: Table routing map (backend/sharding/router.go)

```
1 var tableRoutes = map[string]TableRoute{
2     "Administrator":
3     {KeyColumn: "AdminID",      Placement: PlacementReplicate},
4     "Category":
5     {KeyColumn: "CategoryID",   Placement: PlacementReplicate},
6     "Member":
7     {KeyColumn: "MemberID",     Placement: PlacementPartition},
8     "WishRequest":
9     {KeyColumn: "WishRequestID", Placement: PlacementPartition},
10    "Listing":
11    {KeyColumn: "ListingID",     Placement: PlacementPartition},
12    "ListingImage":
13    {KeyColumn: "ImageID",      Placement: PlacementPartition},
14    "Offer":
15    {KeyColumn: "OfferID",      Placement: PlacementPartition},
16    "Transaction":
17    {KeyColumn: "TransactionID", Placement: PlacementPartition},
18    "MessageThread":
19    {KeyColumn: "ThreadID",     Placement: PlacementPartition},
20    "Message":
21    {KeyColumn: "MessageID",    Placement: PlacementPartition},
22    "Notification":
23    {KeyColumn: "NotificationID", Placement: PlacementPartition},
24    "Watchlist":
25    {KeyColumn: "WatchlistID",  Placement: PlacementPartition},
26    "Report":
27    {KeyColumn: "ReportID",     Placement: PlacementPartition},
```

```
28     "Rating":
29     {KeyColumn: "RatingID",     Placement: PlacementPartition},
30 }
```

Administrator and Category are replicated to every shard because they are small, read-only reference tables that are joined frequently during listing and user lookups.

2.4 Migration Script

Migration is performed by `scripts/migrate_shards.py`. The flow is:

- (1) Checks the three shard databases exist or not.
- (2) Read each row from the source CampusTradingB database.
- (3) Route partitioned rows to the owning shard using the modulo rule.
- (4) Copy replicated rows to every shard.
- (5) Commit all shard writes.
- (6) Print a per-table row count summary.

Listing 3: Migration routing logic (scripts/migrate_shards.py)

```
1 def copy_partitioned_table(source_cursor, shard_connections,
2                             router, base_database, table):
3     columns, rows = fetch_primary_rows(source_cursor,
4                                         base_database, table)
5     for row in rows:
6         routing_value = value_for_routing(table, row, columns)
7         shard_id = router.shard_id_for(routing_value)
8         # insert into shard_connections[shard_id] ...
9
10    def copy_replicated_table(source_cursor, shard_connections,
11                              base_database, table):
12        columns, rows = fetch_primary_rows(source_cursor,
13                                            base_database, table)
14        for shard_id, shard_conn in enumerate(shard_connections):
15            # insert all rows into every shard ...
```

2.5 Verification

Verification is handled by `scripts/verify_shards.py`, which performs two checks:

- **Total integrity:** the sum of row counts across all three shards equals the source table's row count — no rows are lost.
- **Uniqueness:** no primary key appears on more than one shard (no duplication).

All partitioned tables passed both checks after migration. Replicated tables are verified to have identical row counts on all three shards.

3 SubTask 3 — Query Routing

3.1 Connection Architecture

The backend opens one connection pool for the base database and one per shard. Shard database names are derived from the base name, so the same DSN template is reused with only the database component replaced:

Listing 4: Shard connection setup (backend/db/db.go)

```
1 sharding.Configure(baseDatabase, sharding.DefaultShardCount)
2 Shards = openShardConnections(baseDatabase,
3                               sharding.DefaultShardCount, dsn)
```

3.2 Shard ID Computation

Listing 5: Router.ShardIDFor — O(1) shard selection

```

1 func (r *Router) ShardIDFor(recordID int) int {
2     if r == nil || r.shardCount <= 0 {
3         return 0
4     }
5     shardID := recordID % r.shardCount
6     if shardID < 0 {
7         shardID += r.shardCount // handle negative IDs safely
8     }
9     return shardID
10 }

```

3.3 Routing Patterns

3.3.1 Single-record lookup / write.

The backend extracts the primary key from the request path or body, calls `ShardIDFor`, and connects directly to the owning shard. No other shards are contacted.

3.3.2 Browse-style (fan-out) queries.

For queries without a known primary key — such as listing all active listings — the backend queries all three shards in parallel and merges the results in application code:

Listing 6: Fan-out across shards for browse queries

```

1 func fetchListingRowsAcrossShards(ctx context.Context,
2     baseWhere string, args []interface{}) ([]listingRow, error) {
3     {
4         query := `SELECT ListingID, SellerID, CategoryID, Title,
5             Description, AskingPrice, IsNegotiable,
6             ` + "`Condition`" + `, Status,
7             CreatedDate, LastModifiedDate,
8             WishRequestID
9             FROM Listing l` + baseWhere
10
11         var rows []listingRow
12         for _, shardDB := range appdb.AllShardConnections() {
13             shardRows, err := shardDB.QueryContext(ctx, query, args
14                 ...)
15             // ... append shardRows to rows ...
16         }
17         return rows, nil
18     }
19 }

```

3.3.3 Related-record routing (offer workflow).

For operations centered on a listing, the listing ID pins the main workflow to the listing's shard so multi-step actions stay colocated. Example:

Query type	Routing behaviour	Example
Single lookup	Direct to owning shard	GET /listings/{id}
Single insert	Allocate the new ListingID, then write to the owning shard	POST /listings
Related write	Route via the parent listing shard	POST /listings/{id}/offers
Browse / range	Fan-out across shards and merge results	GET /listings

4 SubTask 4 — Scalability & Trade-offs Analysis

4.1 Horizontal vs. Vertical Scaling

Vertical scaling upgrades one machine with more CPU, RAM, or disk. It is operationally simple and requires no application changes, but it converges on a hard ceiling — a single server can only be upgraded so far, and every write is still a single bottleneck.

Horizontal scaling with shards distributes both data and query load across multiple independent database nodes. Capacity grows by adding shards rather than by upgrading one server.

The Campus Trading App's three-shard design is a concrete example: each shard handles one-third of the partitioned table rows and one-third of the single-record lookup traffic, so the system can be extended to more shards without changing the application logic beyond the shard count configuration.

4.2 Consistency

Each shard is a standard InnoDB database, so reads and writes within a single shard are fully ACID-transactional and consistent.

The system does *not* provide a global transaction across all shards. Two specific consistency risks exist:

- **Cross-shard reads:** a browse query fans out to all three shards sequentially. If one shard is mid-transaction when it is queried, the merged result may reflect a partially committed state.
- **Replicated tables:** `Administrator` and `Category` are written to all three shards independently. A write to one shard that fails before reaching the others leaves the shards temporarily inconsistent until the next migration or manual reconciliation. But this case will never happen as these tables can't be updated through the application.

Consistency boundary

For the campus marketplace workload these inconsistency windows are acceptable: the short fan-out merge window is unlikely to coincide with an in-progress transaction on that specific row. A production system would require two-phase commit or a distributed coordination layer to close this gap.

4.3 Availability

If one shard becomes unavailable, only the records mapped to that shard are affected. The other two shards continue to serve their own data normally. For the current modulo-3 scheme, a single-shard failure degrades approximately one-third of the data — but only the rows whose `record_id mod 3` maps to the failed shard.

The implementation does not include automatic failover, replica promotion, or retry logic. A failed shard surfaces as a connection error to the API handler, which must handle it explicitly to avoid cascading failures.

4.4 Partition Tolerance

The design achieves basic partition tolerance by keeping shard boundaries independent: a request that needs data from shard 1 never touches shard 0 or shard 2, so a network partition between two shards does not affect the third.

5 Observations and Limitations

5.1 Observations

The modulo shard key produced a balanced distribution for the current data set. Verified post-migration counts stayed within one row of perfect balance across all three shards for every partitioned table, confirming that the routing rule worked as intended.

The migration process was straightforward. Partitioned tables went to a single owning shard and replicated tables were copied to

all shards; the verification script confirmed that totals matched the source database and no primary key appeared on multiple shards.

One practical observation is that shard-aware access had to be explicit throughout the backend. Single-record lookups and writes were routed directly and required minimal change from the Assignment 2 handlers. Browse-style queries required more effort: fan-out, per-shard result collection, and in-memory merge and re-sorting of the results. For queries with `ORDER BY` and `LIMIT`, the merge step must collect a superset from every shard and then apply the final sort and limit in application code, which is less efficient than the equivalent single-database query.

5.2 Limitations

- **No cross-shard transactions:** a write touching rows on two different shards cannot be made atomic without a two-phase commit protocol, which is not implemented.
- **Fan-out cost:** range queries, admin reports, and any browse query must hit all three shards and merge results in application memory, making them more expensive than a single-database equivalent.
- **No automatic failure recovery:** if one shard goes offline, the application has no built-in retry, failover, or read-from-replica logic.
- **Live rebalancing is not supported:** the modulo key is fixed at `num_shards = 3`. Changing the shard count requires a full re-migration; there is no online resharding capability.

- **Replicated table staleness:** Administrator and Category updates must be written to all three shards manually or via the migration script. A missed write leaves one shard with stale reference data.

6 Author Contributions

All five members contributed equally to the sharding design, implementation, testing, migration scripting, and report writing.

- **Bhavik Patel** — Shard key analysis, distribution verification, and skew analysis.
- **Hitesh Kumar** — Go router implementation, shard connection management, and fan-out query logic.
- **Jinil Patel** — Frontend adaptation for shard-aware API endpoints.
- **Pranav Patil** — Shard DDL scripts, migration script (`migrate_shards.py`), and data integrity checks.
- **Sibtain Raza** — Verification script (`verify_shards.py`), scalability trade-offs analysis, and report editing.

All members jointly performed system integration, end-to-end testing, and final report preparation.

Acknowledgments

We all acknowledge the Course Professor Yogesh Meena and all the respected TAs for providing clear understanding of the assignment and solving doubts at each stage.